Agent 时代沙箱系统的实践

标准录音 11录音 · 2026/10/23 12:01 - 12:33 · 整理于 2026/10/23 19:54

一、Agent 时代沙箱面临的三大问题

Agent 自主性与安全边界的矛盾

Agent 具有自主性，可以决定下一步去做什么，在干活时需要碰各类数据，因此需要对其做限制。但如果限制得太死，Agent 可能做不了很多事情；放得太松又存在风险——例如把数据库权限给 Agent 后，不知道它下一步会不会把数据库里的数据直接清掉。

生命周期与资源成本问题

大家都希望自己的 Agent 是长期运行的，workspace、会话依赖等都要留得住。但从公司平台视角看，资源成本可能无法接受——不可能为了用户的某次 edit，让 Agent 7×24 小时运行，真的去买一个 ECS 放在那里，会造成巨大的资源浪费。

瞬时规模化和环境一致性

在 RL 训练场景中，大多数时候需要拉起几百个上千个一致的环境，在拉起过程中，对整个镜像、依赖缓存、调度都会承受一定的压力。

二、沙箱的三类场景与五类边界

三类产品场景

1. **托管 Agent 运行**：像个人的 personal editor（如缪斯、dot）、年初的龙虾，本质上都是托管在沙箱里运行的；
2. **供应链执行环境**：像 Hugging Face 这类场景下，需要大规模拉起沙箱做训练，沙箱系统本质上是为训练提供环境；
3. **隔离高危操作**：有些 Agent 会做手脑分离，在这种场景下，沙箱去执行一些高危的指令。

五类边界

1. **动作和放行的边界**：管控执行边界，让爆炸半径始终限制在一个沙箱内部，最多用户只会损坏掉他自己的一个沙箱；
2. **网络的边界**：对沙箱来说网络管控非常重要，无论进口、出口，还是横向的访问；
3. **状态的边界**；
4. **身份与审计**；
5. **资源供给和成本**。

内部项目"阿卡"

蚂蚁内部的商业项目叫**阿卡**，取自拉丁语"箱子、盒子"的意思，含义是能把珍贵的物品稳定收纳护在其中的容器。该项目已在蚂蚁内部大规模应用，脱敏后列出几个简单数据：

- 内部能做到**百毫秒级**的沙箱创建延迟，实际生产大概在 50 毫秒左右；
- 日均大概会创建**百万级**的沙箱请求；
- 提供多维度的安全隔离：基于 runD 安全容器、基于 VPC 做的网络隔离、支付和身份的隔离、存储的隔离；
- 网络上做到**100% 审计覆盖**，所有商业流量全量覆盖；
- 通过超卖和纯数据降低资源使用成本。

三、沙箱托管 Agent 的实践

业务视角与平台视角的矛盾

视角	诉求
业务视角	Linux 永远在线（7×24），环境不丢失，不希望运行一段时间后换了沙箱导致 workspace 和依赖不见；任务连续，长周期任务能顺利执行完成
平台视角	低成本，资源充分利用，非活跃沙箱及时回收；需要可中断——平台一定会出现物理机故障、基础设施升级等场景，需要能中断沙箱做平滑迁移

本质矛盾：业务侧永远在线的要求，与平台侧需要"该停就能停下来"的矛盾。做沙箱系统时要考虑让 Agent 能住下来，并且平台能养得起这么多 Agent（内部有 10000 多个 Agent）。

第一个坑：边界划得太严格

刚开始构建沙箱系统时，第一次构建了一个叫 **untrust 区**，跑在里面所有服务都被认为是不安全、不信任的，沙箱和所有用户的 Agent 都跑在里面。上线后遇到问题：用户吐槽平台复杂难用，访问公司办公网不通，需要开白名单做各种操作。

- **踩坑结论**：边界划得太严格，只能承接一些比较简单的业务，只允许访问外网，通不了公司办公网，内部业务用不起来。
- **后续改进**：再划了一个区域叫**办公区**，允许沙箱默认访问公司内部办公服务，所有出口访问经过统一出口网关，在网关层做管控。**边界不是越严越好，而是要划到适当的位置**——untrust 区可放在公司面向 C 端用户的场景，办公区面向公司内部办公类 Agent。

Agent 治理：状态拆分治理成本

第一个是做各种状态的拆分，本质与通算业务没有核心区别。通算业务状态基本都在 DB，所有状态最后都是持久化的，Pod 只是一个随时可回收的计算资源。做沙箱做 Agent 本质上一样——如果把状态都绑定在沙箱上，就很难做各种事情：

- **计算状态**：进程运行时内存，都是实例可回收、可重建的；
- **文件状态**：Agent 的存储大多不是用 DB，都是文件存储，需要持久化、跨实例复用，肯定不是本地文件系统，因为本地文件系统在沙箱销毁后文件无法保存；
- **身份状态**：做沙箱时首先要解决的安全问题是把身份从沙箱里剥离出来，不能把身份信息注入到沙箱——身份在沙箱外部，当网络请求打出去时在请求上注入。

状态从沙箱剥离后，可以独立看存储成本、计算资源成本，实现实际状态可回收，严格控制沙箱成本。

成本治理：快照与 Restore

对运行中的沙箱做**快照**，快照之后可以回收掉用户的计算资源；下次用户需要恢复沙箱时，基于快照做 restore 快速恢复。快照里包含进程、磁盘数据，恢复时直接用快照 restore 方式，业界很多商业产品基本都基于快照这套模式。好处：

- **计算资源的释放**：通过做快照保存用户状态，用户空闲时可释放计算资源；
- **提升启动速度**：创建沙箱时如果经历大量用户初始化交互非常慢，很多业界产品在用户模板里第一次创建沙箱时就保存好快照，以后创建直接基于快照。

平台五项托管能力

模板能力：用户在平台上提交自己的模板，在模板里指定多个镜像，然后做存储配置（用哪个文件系统存数据）、指定规格（如 4C8G）、做网络配置（白名单、黑名单），配置好后基于模板直接创建沙箱。模板支持**版本化**——可以对模板做变更或更新镜像，通过版本做引流达到灰度效果，并可基于模板版本回退。

生命周期托管：沙箱的创建、运行、自动暂停、回收、恢复、继续都由平台直接控制。

平台四类沙箱：默认支持代码执行器、浏览器、电脑以及移动端（安卓沙箱），并支持用户自定义沙箱。对不同底座的沙箱做统一抽象、统一接口操作，整个底座统一基于安全容器做隔离与资源约束，统一提供快照恢复和挂载。

成本治理实践：预热池到缓存池

方案	机制	问题
预热池	基于模板创建预热池（如 100 个），后台 standby 100 个沙箱基于模板构建、网络不通、随时待用；创建时直接取出一个、改网络配置马上交付，20–30 毫秒即可交付	空闲资源占用——业务无论有没有流量都会占用这么多资源；需按业务峰值评估池大小，池子用完就进入冷启动
缓存池	仍有少量 standby 沙箱，但池子**用一个补一个**，像队列一样从头取一个、尾部系统快速补一个；池子可做得很小，大小根据沙箱组速度和业务并发量算出，结合快照技术快速恢复补沙箱	—

内部实际效果：从预热池切到缓存池后，成本节约了 96%。内部业务原来面向公司内部员工需要几万个沙箱，上线该能力后按峰值只要缓存池里几百个沙箱即可。

生产问题：系统级 Skill 被绕过修改

业务场景中会起很多沙箱，沙箱里跑 Agent，会定制**系统级 Skill**（由平台发布、不是用户定义，默认装载到 Agent 中）。每个沙箱每个用户都会加载这些数据，Skill 文件内容不能被修改。

- **起初实现**：在文件目录里控制权限，设为只读不能改；
- **上线后发现被绕过**：用户镜像启动沙箱时直接用 root 权限启动，root 直接绕过权限控制。带来的问题——某用户进入沙箱是 root 权限，把系统 Skill 改了，其他用户再创建沙箱加载的系统 Skill 就是被改过的，导致 Agent 起来的服务错乱；
- **改造方案**：从文件系统权限级别改到**挂载控制**——从沙箱挂存储时支持只读挂载，从内核层面避免沙箱对某些文件的修改。业务侧保留可写链路，有两条：业务侧通过可写化做系统 Skill 的发布更新；沙箱侧面向 C 端用户，用户修改文件被系统内核直接拒绝。

让 Agent 住下来且养得起的核心要点

1. 控制好网络边界，放到合适区域；
2. 对状态做拆分，拆分了状态才能做更好的成本治理；
3. 计算的恢复；
4. 通过超卖、空闲回收、缓存池提升资源利用率。

四、沙箱作为大规模环境供应链

RL 训练场景中的环境等待问题

很多时候是**在等环境而不是等 GPU**。拉起 RL 训练时要大规模拉起沙箱，需要大量拉取镜像、做依赖安装、调度、挂载，读取这些内容都会打到网络带宽上。环节中任何一步没完成，就不是在等算力，而是**算力在等环境**。

生产事故与反思

某次晚上一个业务做 RL 训练，训练数据在 OSS 上，拉起训练时顺势拉起大批量沙箱集中读 OSS 数据，直接把带宽打满，导致丢包和网络问题，影响其他业务。当时立即上线做治理，找到业务关掉 RL 训练。后续 Action：

- 根据不同业务场景做隔离，在公共依赖上（镜像中心、网络带宽、出口网关）按不同业务类型（在线推荐业务、训练业务）做隔离、带宽扩容、各类监控，解决共享链路上共享节点相互影响的问题。

镜像链路拆分与优化

不同场景启动要求不一样。缓存池本质是提升单个沙箱的交互效率，但 RL 场景需要大批量沙箱交付，更多运用**快照启动**。做了镜像加速和内存快照加速启动。内部用内部镜像测试：没开镜像加速拉起需要两分多钟，开启后可以做到秒级加速。

调度优化

- 大规模拉起时需要镜像已经在物理机节点上下载好，做好**镜像缓存**，找到合适的一批节点把沙箱调度上去，做各种缓存命中预热操作；只有这些操作不命中时才降级到通用节点；
- 做好**配额管控**：用户有多少配额可以跑训练，配额不足时直接拒绝，而不是给部分资源，导致不同用户都出现容量不足。

可观测：环境交付五阶段

把整个环境交付拆分为五个阶段：**资源准备 → 调度 → 挂载 → 启动 → 就绪**。对五个阶段都做指标监控，定义关键指标和核心失败原因，出现问题能快速定位。

环境供给核心要求

要求	说明
可复现	基于一个沙箱做修复、打快照，基于快照 fork 出大规模沙箱，做到复现效果
可加速	通过镜像加速、镜像分层等机制提升交互效率
可调度	面向 RL 场景做调度优化
可观测	把整个链路串起来，针对性优化训练场景下各阶段耗时

五、沙箱作为高危动作执行器

高危操作场景面临的问题

- Agent 要执行一些操作时，应该由谁做决策和判断；
- 操作执行的结果怎么共享；
- 执行身份怎么输入；
- 怎么做端到端的审计——谁在里面做了什么。

实践方案

- **统一记录、身份凭证与安全审计**：有策略中心。执行操作时在策略中心做执行动作分类——调 MCP 是查询，是低风险操作；执行一段代码是高风险操作。基于操作风险判断决定操作能否在 Agent host 上直接发起，还是放到沙箱里执行，或者高危操作需要走人工审批流程确认后才能运行。
- **状态保存与结果共享**：大量场景用 OSS、NAS 等存储系统做状态保存。Agent 除了给 response 告诉动作完成，过程中会产生副作用——例如让 Agent 做一个 PPT，PPT 会挂到外部存储设备上，A 沙箱销毁后 B 沙箱拉起时，可以通过沙箱 ID 把 A 沙箱的存储挂回来，拿到 A 沙箱的文件，保证结果不丢失。
- **全链路身份与审计**：从沙箱请求进来到网络操作出去，做到每次请求都可以归因。平台上有身份托管，每个租户操作 OSS 时 AK、SK 都托管在平台上，Agent 拿不到身份、只有占位符；在网络出口做流量劫持，在流量里注入真正需要的 AK、SK。与公司内部平台集成，每个人需要自己的身份和授权，整条链路上交互用短 token 做验证。
- **受控边界**：高危动作操作都需要进受控边界，核心两个点——**入口网关**和**出口网关**。

整体架构总结

左侧是产品层，核心功能是模板管理。整体分层：

1. **接入层**：提供 SDK、HTTP、MCP 接入方式；
2. **沙箱网关**：所有沙箱请求通过网关进入沙箱；
3. **沙箱实例层**：提供五类沙箱能力；
4. **统一调度层**；

5. **安全底座**：基于 runD 的安全容器，物理机上有网络代理，网络出口有统一安全网关；
6. **可观测能力**：贯穿全链路。

三类场景核心要求对比

场景	核心要求
托管 Agent	环境不能丢、会话不能中断、成本可控；生命周期、持久化存储、审计都需要管控
训练供应链	核心诉求是批量供应能力——能否在短时间内供应大量环境
高危操作执行	风险分级决策、统一身份与审计、入口/出口网关受控边界

由 NEXT AI 生成，内容仅供参考